

Predicting Video Game Sales Based on Pre-Release Features Using Machine Learning

Cook
Fordham University

Brooklyn, New York
jmcook5520@gmail.com

Abstract— This project sought to answer whether similarity to past video games could improve a predictive model of video game success utilizing machine learning techniques. A baseline logistic model was created, then compared to models that utilized a sentence transformation neural network to transform game descriptions into comparable numeric vectors. The models, while not very strong in their predictive capabilities, did display that the semantic information transformed from the descriptions did contribute to predictive power, especially when a principal component analysis was performed on the vectors. The project displayed that the game descriptions do hold some relationship to the video games' success.

I. INTRODUCTION

Video game development is a high-risk endeavor, with large amounts of time, effort and money put into creating games that often don't succeed. In an unforgiving market with constant fluctuation, a predictive data model on video game success would be a substantial tool for the decision making process of game developers in both how to modify a work-in-progress and whether to go forward with the creation of a game at all.

However, predicting market success is inherently a difficult task, as consumer practices are impacted by many non-calculable factors, and video game metadata contains much noise that can interfere with a data model.

This study investigates whether machine learning methods can improve the prediction of video game success using only features available before the game's release. Primarily, this project utilizes a sentence-embedding model to compare game descriptions to previous successes in order to improve a baseline model's predictive performance. This work tests multiple models, both with and without the description features to evaluate the effectiveness of these features.

II. RELATED WORK

A. Machine Learning Methods

Machine Learning methods have improved on classification capabilities of predictive models, particularly in the way of a logistic regression model, and are useful in parsing similarity-based features. This project draws from logistic regression models, Ridge and LASSO regularization techniques, Support Vector Machines and Principal Component Analysis.

B. Sentence Embedding Model

Python's SBERT library utilized natural language processing sentence transforming models that map sentences to vectors of numerical values. Utilizing this model to evaluate

semantic relationships between descriptions is a great way to track similarity to map only predictive models.

III. DATASET

A. Data Source

The data this model is trained on is a Kaggle dataset of games from the PC platform Steam [1], with 47 column features of game data including game descriptions, estimated owners, genre, tags, and more. There are over 114,000 entries in this dataset, each representing one game. The original dataset contained inconsistencies and formatting issues that required preprocessing prior to model training.

B. Feature Description

The features in this set fall into numerous categories. The focus of this project is pre-release features, so features like reviews were discarded. There are metadata features such as release date, price, and developer, alongside descriptive features like genre, tags, and game description.

C. Target Variable

The target variable for this classification model is based off of the "estimated-owners" column. Steam does not provide exact numerical data on sales per game, so each game is categorized into a range in which its sales fall. These ranges were simplified by representing them by their midpoint. The buckets were then categorized as "hits" when at or above the 50,000-100,000 estimated owners bucket. This is a low threshold in the categories, but when separated, the "hit" to "non-hit" ratio came out to 12.4:87.6. This class imbalance is reflective of the video game market, most games do not reap the success they sought out to achieve. The target variable is a binary classification of "hit" and "non-hit", which is how these categories will be referred to going forward.

D. Data Preprocessing

The first step was basic dataset cleaning, including removal of games missing values for the relevant features for the model. Not only is this important for building the model more effectively, but it also prunes the dataset of "spam" games uploaded to Steam that don't match the effort and proper development of games that this model would have use cases for.

Given the importance the game descriptions held in creating the similarity features, it was important to clean the text of the descriptions beyond missing values. For this, punctuation and capitalization was removed to increase the effectiveness of the sentence transforming model. The next important cleanup step for the descriptions was removing terms and phrases like "Game of the Year Edition" or "Collector's

Edition” from the descriptions. These are noisy phrases the strongly correlate with hits, as they only release if the game is already very successful. The games were not fully removed from the dataset, as their inclusions would contribute to the modeling of a later engineered “developer success” feature.

The model ran into a large issue that required a change in the set of features modeled. Steam games come with tags, which are one to two word descriptors. The tags were initially used as features alongside the one-hot encoded genre features to map game metadata, but amidst the project it was discovered that tags, previously assumed to be developer listed, were actually user submitted, making them post-launch features. Their removal resulted in a 0.1 decrease to the model’s F1 score, but this made for a more honest model for the stated goals. This was impactful data leakage that was important to remove.

Features were engineered for model use. Due to the fact that games released by previously successful developers sell more due to name recognition, a feature tracking the developer’s ratio of “hit” games over the number of total games released by them was designed, alongside a feature that flatly tracked the number of games the developer has released. This was done by ordering the dataset in chronological order by release date, then iterating through the set and tracking previous games for each developer with a Python dictionary. Through this, a dev “success score”, a feature tracking whether the developer has had at least one hit before, and a feature tracking the number of games the developer has released previously engineered. The developer experience feature had a logarithmic transformation applied due to the potential large number of games released in the hundreds compared to most developers that remain in the single digits. The dev success feature ended up being one of the most heavily weighed features for the model.

E. Class Imbalance

As noted before, the hit threshold that was decided upon resulted in a classification ratio of 12.4 hits to 87.6 non hits. Due to the very small positive rate, accuracy was not a good metric by which to measure the model’s success. To test model effectiveness, this project aims to maximize its F1 score, with some preference towards precision rather than recall, as a low false positive rate would be conducive to helping developers know whether their game has a high chance of being successful.

IV. METHODOLOGY

This project’s goal was to discover if the similarity features provided by the game descriptions could give significant insight to the data model that improved its predictive quality. This required multiple models to be compared to each other.

A. Baseline Logistic Model

The baseline model to be compared to is a simple logistic regression model implemented with scikit-learn [2]. This model was chosen for its efficiency and predictive capability for classification purposes. Logistic Regression models the probability a sample falls into a category with a linear decision threshold and a sigmoid activation function. Numerical values were standardized with StandardScaler to normalize feature distributions, and a balanced class weighting was applied due to the inherent class imbalance in the dataset.

The initial baseline model was trained with non similarity pre release metadata features, including pricing, release date, the engineered dev features and one hot encoded genre features. This model was the reference point for later models to see if there would be significant improvement.

B. Sentence Embedding

To capture the semantic relationships between name descriptions, the SentenceTransformer implementation of the all-MiniLM-L6-v2 transformer model was applied iteratively to the game descriptions column. The sentence embedding model is a neural network that transforms text into dense numerical vector representations [3]. As mentioned in Section II D, noisy elements of the description texts were removed prior to applying this model.

C. Similarity Feature Engineering

The similarity feature was created using cosine similarity applied to the sentence embedded vector of a current game with prior released hit games.

Similarity comparisons were applied in release order so as to prevent data leakage. As with the rest of the model, this feature had to be engineered such that it only contained information available pre game release. Several features were engineered and tested, including a simple similarity to the five most similar previous hits, a feature tracking recency to similar hits, and two interaction features, one to developer success and one to the recency metric.

D. Principal Component Analysis

While the similarity features represented a more targeted usage of the sentence embedding, the model produced vectors with a large number of dimensions, and compressing them into four feature columns would naturally result in the loss of semantic data and relationships.

To accommodate for this, a principal component analysis was applied to the sentence embeddings, implemented with scikit-learn [2]. PCA reduces the dimensionality of a high dimension set like the embeddings but finding a subset of components with the greatest variance within the data. Multiple component amounts were tested to maximize the predictive power of the model.

The PCA-reduced embeddings were concatenated to the previously used features to add the impacts of the richer semantic results of the embedding while limiting the noise added to the model.

E. Alternative Models

For further tests, alternative models were evaluated. An L1 LASSO regression model was tested for a sparser feature set to see if any features were adding noise to the model. A Support Vector Machine was also tested to see if there was significant non-linear relationships amongst specifically the description data that lent itself to a better prediction model.

These models had mixed results, and showed that model complexity did not inherently improve the success of this model. The SVM model’s worse F1 score displays that the dataset did not contain strong non-linear relationships for the model to derive better predictive performance from.

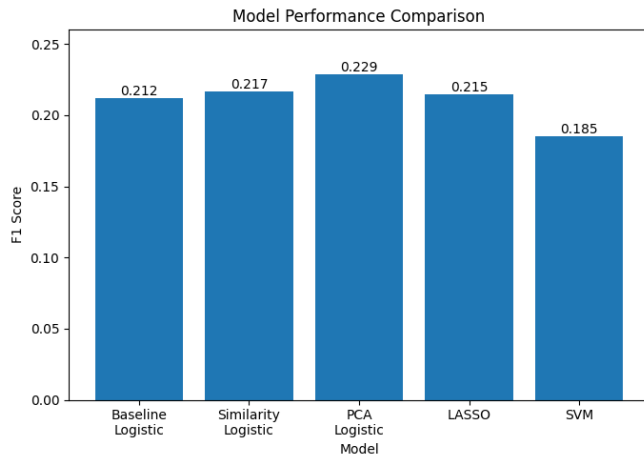


Fig. 1. Alternative models F1 scores compared to main models.

F. Evaluation Metrics

While precision was to be prioritized over recall, the primary metric to be evaluated was the F1 score, as simply increasing the classification threshold to increase precision resulted in a small number of positives being predicted altogether.

To track this threshold balancing, a Precision Recall Curve was plotted for the baseline and final model to compare their average precision scores, evaluating each models ability to rank successful games over unsuccessful games.

V. RESULTS

The models all resulted in low F1 scores hovering around .2, but the small differences in results between models and the lack of difference overall hold insights to the nature of the video game consumer market and feature importance.

A. Baseline Performance

Initial baseline testing resulted in a low F1 score of .212, indicating that, while some positives were successfully predicted, substantial overlap remained in its prediction of false positives and false negatives. The data was split into a train/test split of 7:3, with the test data all chronologically after the training data.

Looking at the coefficients per feature, developer success was the strongest indicator, which is expected when considering the real life implications. However, this means that the model is very reliant on developer's previous success to predict in this model.

The baseline model displays the inherent difficulty of predicting market behavior with solely pre release features without significant information about the product itself.

B. Similarity Feature Results

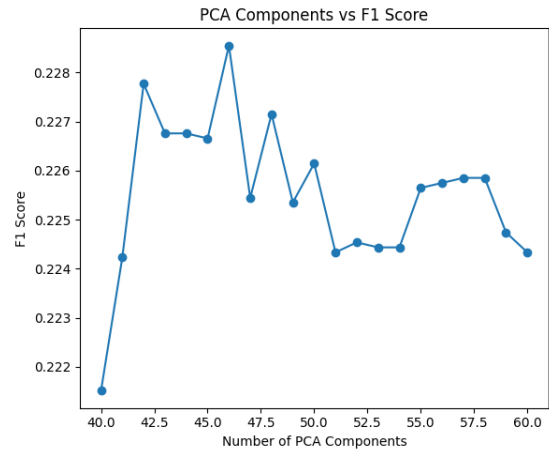
The engineered similarity based features resulted in modest improvement over the baseline, with an F1 score of .21. These did contribute positively to the F1 score, but only by .003.

The semantic relational data lost in the compression of the embeddings into four features resulted in limited improvement from the baseline. Despite this, the coefficient placing of the similarity to past hits score, only trailing behind developer success, in addition to the positive change in prediction does show that there is a meaningful relationship in game success and similarity to previously successful games.

C. PCA Results

The strongest result of this project came from the model trained with using features from a principal component analysis on the sentence embedded descriptions. After testing component amounts between 40 and 60, the F1 score was notably decreasing going outward from those number of components. The best results were between 45 and 50, with 46 components resulting in an F1 score of .229, a much larger increase compared to just the similarity features.

These results showed that the semantic information within



the test descriptions do hold relational data impacting the ability to predict video game success, and is most effectively found when more dimensions of the embedding are retained through PCA.

Fig. 2. PCA component numbers vs. F1 score.

TABLE I
MODEL PERFORMANCE COMPARISON

Model	Feature Set	F1 Score	Average Precision
Logistic Regression	Metadata Baseline	0.212	0.12
Logistic Regression	Similarity Features	0.217	0.12
Logistic Regression + PCA	Metadata + PCA Embeddings	0.229	0.15
LASSO Regression	Metadata Baseline	0.215	0.12
Support Vector Machine	Metadata Baseline	0.185	0.11

D. Precision-Recall Analysis

Every model so far had accuracy scores over .95 due to the large class imbalance found in the dataset. As such, this project's results are derived from the precision, recall, and F1 score. The Precision Recall Curve displays the overall positive prediction ability of each model. A PRC was plotted for the baseline, similarity feature, and PCA models. The PRCs provided their Average Precision (AP) scores, detailing the models ability to rank hits over non hits.

The test data has a hit to non hit ratio of 3.3:96.7, much smaller than the full dataset. Were someone to rank the test set randomly, the average precision score would land around .03, but even the baseline model resulted in an average precision score of .12, along with the similarity feature model. The PCA model took this score up to .15, a significant improvement especially compared to random guessing. These results suggest that semantic relational data extracted from game descriptions contributed meaningful predictive signal to the classification task.

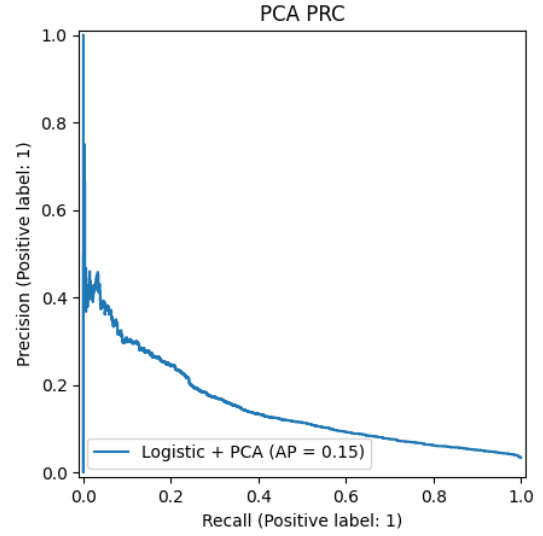
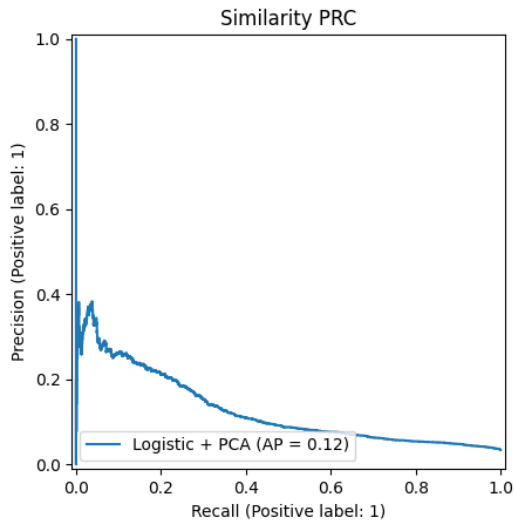
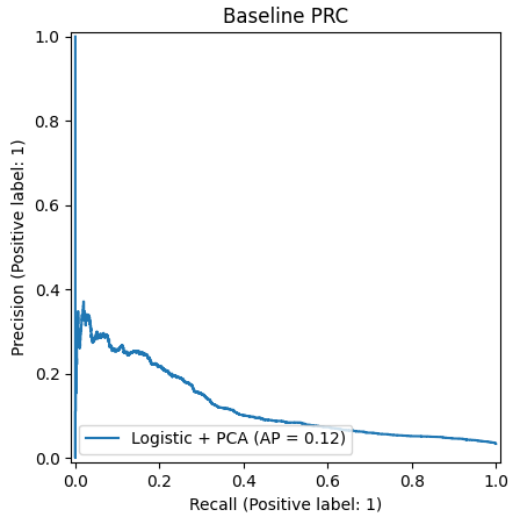


Fig. 3. Precision-Recall Curves comparing Baseline, Similarity and PCA Models.

E. New Developer Test

A consideration was made after the running of the PCA model. Consistently throughout each model, developer success was one of the strongest features, with the highest coefficient across all features with exception to the PCA features. Considering positive developer success is a strong signal for hit games, a test was run to see how the model would perform on a test set with only games made by developers with no previous success.

TABLE II
FEATURE IMPORTANCE FOR SIMILARITY MODEL

Feature	Coefficient
dev_success	0.458
sim_to_past_hits	0.196
Free To Play	0.187
dev_had_success	0.170
RPG	0.159
Simulation	0.108
Strategy	0.102

The test confirmed the concern, when previously successful developers were removed, the PCA model's F1 score dropped to .12 (after threshold tests). This indicated that dev_success was doing a significant amount of the successful predicting for the positive results. However, the PCA feature coefficients and

the PRC mapped for the new developer test display that this does not invalidate the findings of the project.

The AP for the new developer test on the PCA model shows a .03 improvement over random ranking, matching the PCA model's .03 AP improvement upon the baseline and similarity models. This, in addition to the `sim_to_past_hits` feature's ranking in the coefficient sizes upholds the conclusion that the semantic information in game descriptions does have a positive relationship with predicting game success.

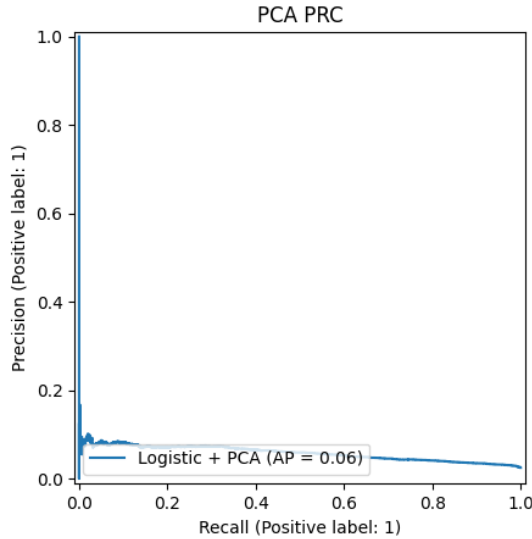


Fig. 4. Precision-Recall Curve for new developer test.

VI. DISCUSSION

A. Difficulty Predicting Video Game Success

Predicting any market is a difficult challenge. For this project, the modeling was limited by the available data for games, relying on descriptions written by the developers rather than any actual in game information.

Many hidden variables impact game success, from marketing budgets, marketing procedures, whether online influencers play it for their audiences, and connection to popular IP outside of the video game industry. There are also economic impacts that affect the average consumer's purchasing patterns.

Limiting to pre-release features makes this inherently a difficult task, and a low output model is unsurprising, even with the sentence embedded features. However, these features did display improvements, which was the question this project aimed to answer.

B. Model Complexity

Throughout this project, model complexity was increased in hopes of finding some non-linear relationships to heavily improve the model. However, a tested SVM model performed worse than any of the other models. This displayed that the dataset, at least amongst the features utilized for this model did not contain strong non-linear bounds that improved the classification performance. Honing the features with PCA was significantly more helpful than simply increasing the

complexity of the model. Machine learning models are not a catch-all for model improvement, some predictive models will only improve from engineering its features better. Machine learning models are very impressive, and on the surface seem like a surefire way to improve the predictive performance of data modeling projects, however, engaging with the features that are utilized and processing the data holds far more importance than simply increasing model complexity.

C. Importance of Developer History

Developer history was the strongest feature in this model from the start, as expected as this is what prompted its inclusion. Testing confirmed that this was causing an unseen skew in the model results, with a true positive rate that primarily came from games made by previously successful developers. When the model was run on a test set with only games made by developers without a previous success, the model performance plummeted.

This shows a great importance in data modeling on thinking through the real life factors that may not easily be seen in the numbers. This catch, while not detrimental to the results of the comparative questions asked by this project, did confirm that the model trained is not well equipped to help developers determine whether a game idea has success potential. However, it does further solidify the conclusions drawn about the positive impact of semantic data from game descriptions having some, even if small, impact on the predictive capabilities of the model.

D. Limitations

This project was limited by its goals, specifically in its commitment to utilizing only pre release features. The dataset was also limiting, with very noisy metadata that is not derived for the purposes of predictive modeling. Even the target variable was not an exact value for each game, simply ranges that the estimated values fell into. With all of this in mind, the model still managed to produce many insights in regards to whether there were relationships between the semantic description data and the success of the games.

The model is also limited by its ability to actually derive gameplay information. The description similarity check is an attempt at gathering this information, but descriptions vary from actually explaining gameplay features to just describing the basics of the game's story. While this is still relevant for the purposes of this project (game stories that are similar to stories of very successful games can benefit from them in the same way they can from similar gameplay), being limited to one or two paragraphs in which a developer has to prioritize what they believe to be the most interesting way to explain their game, games are layered with countless features that appeal to very large and varied spans of consumers. The ability to map far more features than what is provided in the description would be a good test beyond this project. Initially, the user tags fulfilled this role, but seeing as those were post-release, they had to be removed. Utilizing trailers for previously released games with a machine learning model that derived tag-like features and compared them to game success could make the predictive tool possible for developers. However, this would require making a far more advanced set of models, and this project had to be limited in its scope, as it was done for an academic course with a deadline at the end of the semester.

This project was also limited by the singular threshold for what is declared a hit. For different developers, success can look very different. For a small indie developer producing their first game, a game achieving 20,000 sales could be a massive success, while an established triple-A developer with many successes under their belt could see even 100,000 sales as a failure. In this vein, lacking access to game budgets weakens this project, as success is very different based on the development costs of the game. While the probability threshold is easy to modify, the actual “hit” threshold changing would necessitate a re-run and re-modeling, or a creation of multiple different models. It was also limited by the lack of exact data on game sales through steam. The buckets added obscurity to the intended target variable, and while still functional for a binary hit/non hit classifier, limited the models ability to predict success of the games more specifically.

Another large limitation is the difficulty in collecting data on the games exposure prior to release, specifically by social media influencers. Video game developers often do a lot of outreach and paid sponsorships with popular video game streamers and content creators to market their game and garner interest. This, like developer success is a large indicator, and a user ready model could be used to recommend to developers what creators their game would benefit the most from. Like the developer success feature, this would likely need a separate set of models trained that do not include this as a feature, as it would skew positive results to suggest that games need previous developer success or content creator outreach and declare any games without either as non hits.

E. Future Work

A more thorough work with more time would look into marketing data for video games and the early review and game journalist side of the industry. These are very relevant fields to the questions posed in this project. It would also investigate more complex utilizations of the time of game releases, mapping them to economic trends at the time of release.

Expanding the idea into a range of models to be evaluated and distributed for different levels of developers is also a possible step forward for this project.

A future version of this project could also look to utilize the SteamSpy API [4] to create a dataset tailored to this specific project. The SteamSpy API is a tool that the dataset used to compile its data, and it is possible that the dataset is missing out on available features, as it was not designed for this project’s usage of it.

After the running of the model on a test set lacking previously successful developers, the model was found to be lacking in predictive capabilities based solely on actual features of the games themselves. With the conclusions drawn about the impact of semantic data of the games having a positive impact, further work would aim to improve the predictive capabilities of the model itself, modifying or outright removing the developer success features and finding new data features to train the model on. Beyond the aforementioned SteamSpy API, developing new data gathering tools, using machine learning tools observing video data of the games themselves could be developed to gain more specified information about the gameplay and story of a video game, then turn those into new features to create a stronger predictive model.

Overall, for the scope of this project, the secondary goal of creating a model using solely pre release features that could help developers guide their work in progress games to be more likely to succeed fell to the wayside after the models produced precision and F1 scores below .5. Tag features were removed when it turned out they were user submitted after the games release, however, with some work they could be turned into a large support for a model to help developers. While they are user submitted, tags often do describe the actual elements of the game, and if the metadata contained into the page was retained in the model, with either the sentence embedding model used on the descriptions or a model more specifically designed for short phrases, the developers could enter tags that they would personally apply to their game and utilize the prediction model trained on a feature set that contains the semantic data from tags. While the project did not result in such a model, it did produce many challenges and findings that would help create a model that could successfully do so in the future.

VII.

CONCLUSION

This project aimed to create a predictive model utilizing pre release features of video games to predict the market success of an unreleased game. It also sought to evaluate whether evaluating the similarity between new games with past successful games through semantic similarities in their descriptions would improve the predictive power of the model.

The baseline model returned modest results, but improved compared to random guessing. Handcrafted similarity features, while very small, produced measurable improvement to the model’s results. The model with a PCA on the embedded sentence vectors more strongly improved the model, displaying that semantic information was a signal for games success.

This project displays the importance of feature selection, modification, and model selection. With setbacks like the tags being post release information and developer success skewing the model to prioritizing predicting successful developers to make more successful games, what initially seemed like a much more solid predictive model showed diminishing returns when placed under the scrutiny of the real world considerations of its features. There was also usage of more advanced models that ended up with worse results than the seemingly simpler models. The reality is that complexity does not automatically make for a better model, and in some cases like this one, more time and complexity would need to be applied to the feature set before more complex models performed as well as or better than simpler ones.

Market prediction remains very difficult, but semantic representations of game descriptions did improve the predictive power of the logistic regression model on pre release features.

REFERENCES

1. Artemiy Ermilov Steam Games Dataset, Kaggle. [Online]. Available: <https://www.kaggle.com/>. Accessed: May 2026.
2. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

4. SteamSpy API, "SteamSpy: Video Game Sales and Statistics," [Online]. Available: <https://steamspy.com/api.php>. Accessed: May 2026.